COER UNIVERSITY, ROORKEE

Department of Computer Science & Engineering

Name :- K rishna Sharma

CU240250899

Btech Cse “B”

Roll no. 32

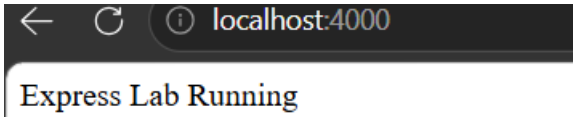
Full Stack Web Development

Lab Sheet 5 — Express.js (Practical)

Objective: Get hands-on with Express.js — routing, middleware, REST APIs, and error handling — by building small, working pieces of code for every core concept.

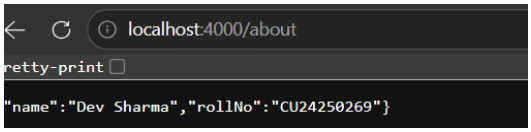
Part A: Project Setup

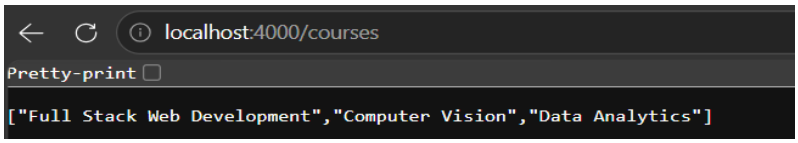
```
const express = require("express");
const path = require("path");
const dotenv = require("dotenv");
dotenv.config();
const studentsRouter = require("./routes/students");
const booksRouter = require("./routes/books");
const resourceRouter = require("./routes/resource");
const app = express();
const PORT = process.env.PORT || 4000;
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);
  next();
});
function adminAuth(req, res, next) {
  if (!req.headers["x-api-key"]) {
    return res.status(403).json({ error: "Forbidden" });
  }
  next(); }
app.get("/", (req, res) => {
  res.send("Express Lab Running");
});
```

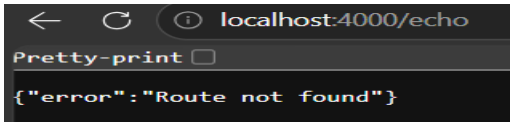


Part B Basic Routing

```
// Task 5
app.get("/about", (req, res) => {
  res.json({ name: "Dev Sharma", rollNo: "CU24250269" });
});
// Task 6
app.get("/courses", (req, res) => {
  res.json(["Full Stack Web Development", "Computer Vision", "Data Analytics"]);
});
// Task 7
app.post("/echo", (req, res) => {
  res.json(req.body);
});
```



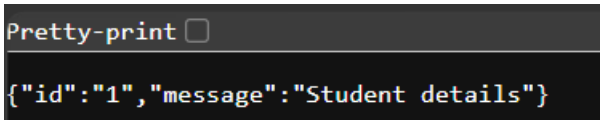




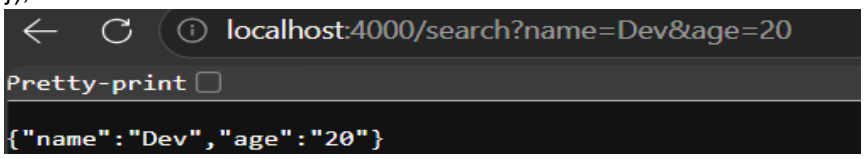
Part C Route Parameters & Query Strings

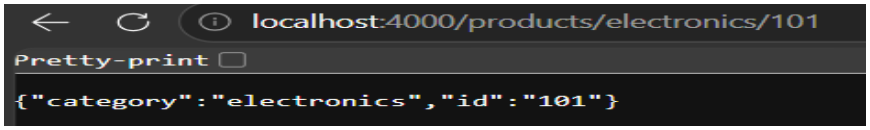
```
// Task 8
router.get("/:id", (req, res, next) => {
  const id = req.params.id;
  const student = students.find((item) => String(item.id) === id);
   if (!student) {
    const err = new Error("Student not found");
    err.status = 404;
    return next(err); }
```

```
  res.json({ id, message: "Student details" });
});
// Task 9
app.get("/search", (req, res) => {
  res.json({
    name: req.query.name || null,
    age: req.query.age || null });
});
// Task 10
app.get("/products/:category/:id", (req, res) => {
  res.json({
    category: req.params.category,
    id: req.params.id });
```




```
});
```

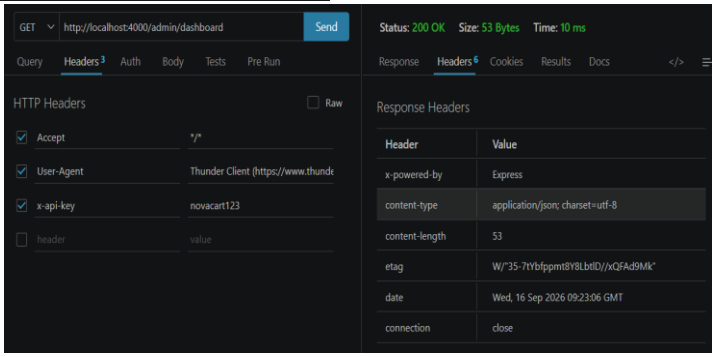


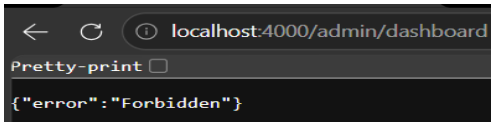


Part D Middleware

```
// Task 11: Global request logger
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);
  next();
});
// Task 12-13
app.use("/admin", adminAuth);
app.get("/admin/dashboard", (req, res) => {
  res.json({ message: "Admin dashboard", data: "Protected data" }); });
```

```
GET /api/students/1 - 2026-09-16T09:04:09.798Z
GET /search?name=Dev&age=20 - 2026-09-16T09:05:19.641Z
```





Part E Working with JSON & Forms

```
#Task 14

app.use(express.json());

app.use(express.urlencoded({ extended: true }));

// Task 15

app.post("/register", (req, res) => {

  const { name } = req.body;

  res.json({

    message: `Registration successful for ${name}`

  });

});

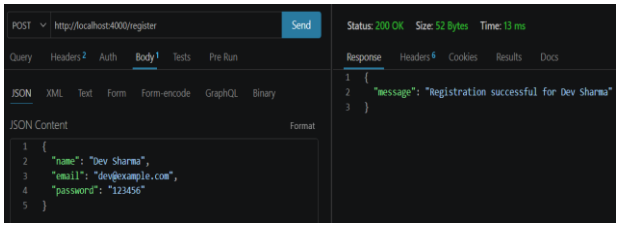
// Task 16

app.post("/contact", (req, res) => {

  console.log(req.body);

  res.json({ message: "Thank you for contacting us." });

});
```





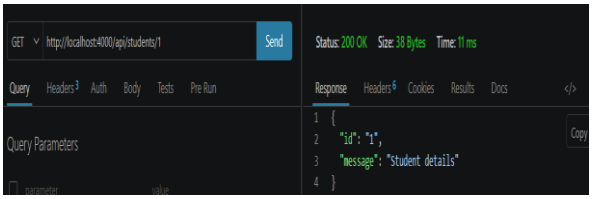

```
Part F  express.Router() – Modular Routes

// Task 17-18

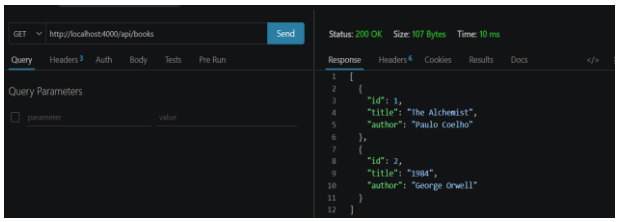
app.use("/api/students", studentsRouter);

// Task 19

app.use("/api/books", booksRouter);
```



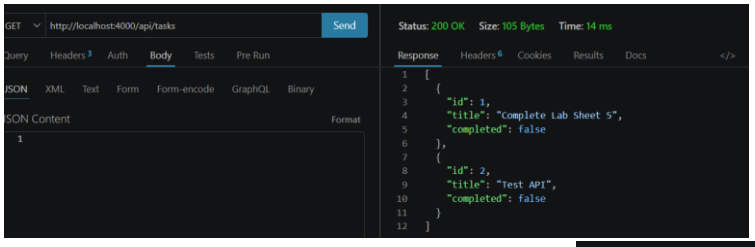




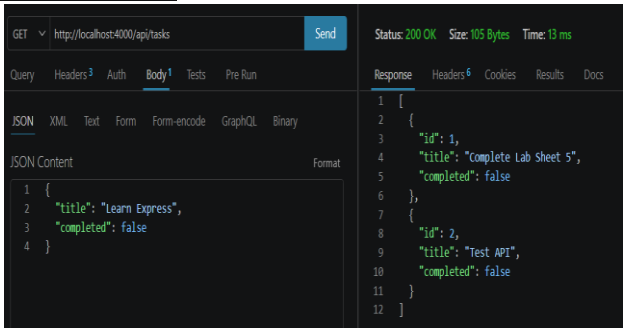
Part G Building a REST API (CRUD)

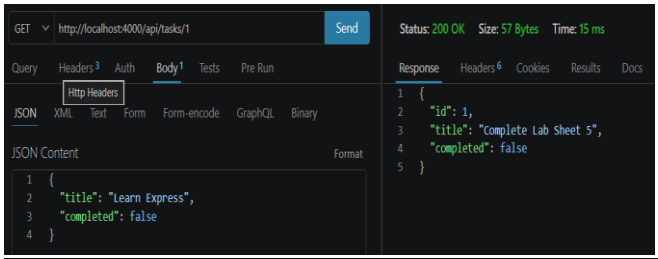
// Task 20

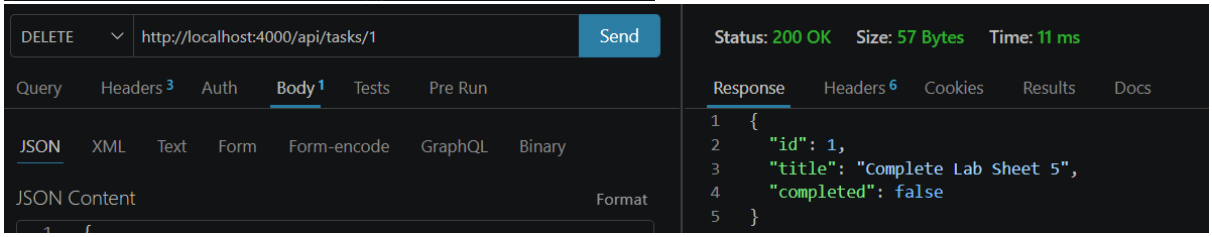
app.use("/api/tasks", resourceRouter);











Part H Error Handling

```
// Task 22: 404 handler
app.use((req, res, next) => {
  res.status(404).json({ error: "Route not found" });
});
// Task 23: centralized error handler
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(err.status || 500).json({
    error: err.message || "Internal server error"
  });
});
app.listen(PORT, () => {
  console.log(`Express server running on http://localhost:${PORT}`);
});
```





Part I Static Files & Environment Config

```
// Task 25
app.use(express.static(path.join(__dirname, "public")));
```

```
[nodemon] starting `node index.js`
Server running on port 4000
```